



# **Artificial Intelligence (CIS2031-N)**

Lab introduction on Functions and Advanced Lists

School of **Computing, Engineering & Digital Technologies**

[tees.ac.uk/computing](https://tees.ac.uk/computing)

# Agenda

## Functions

- Introduce what we mean by functions - reusable pieces of code that return values to the caller.
- Define our own functions in Python and explain their syntax and structure.

## Advanced Lists

- Explore lists in more depth, including useful methods and functions.

# Functions: What are they?

Lecture 3: Part 1

# What is a **function**?

- A **function** is a self-contained procedure or routine - a collection of statements like a mini program within your program.
- They might be:
  - *Built-in*: functions that are built-in come with Python. They're already there for you to use!
  - *User-defined (i.e. custom)*: user-defined functions are functions that we define ourselves as programmers.
- Three functions are defined on the right. What do they do?

```
def triple (x):  
    return x * 3  
  
def square (x):  
    return x ** 2  
  
def is_bigger (x, y):  
    if x > y:  
        return True  
    else:  
        return False
```

# Benefits of functions?



**Convenience:** functions allow us to name a procedure to call when needed!



**Efficiency:** makes the overall algorithm smaller by eliminating repetitive processing.



**Organisation:** breaking a procedure down into named blocks of statements helps code readability.



**Debugging:** makes the program easier to debug and maintain by making it more modular.



**Reusability:** Well-designed functions can be reused elsewhere in other projects.

# Benefits of functions? (Cont.)

- Functions save us *so much time*!
- Built-in functions include `print()` and casts such as `int()`. Imagine if you didn't have these and you had to write out the entire (complex) algorithm to print or cast each time! You'd be there forever!
- Built-in libraries such as `random` contain useful methods such as `random.randint()` and `random.shuffle(list)`. These can save us so much implementation time.
- We can define our own functions if need be using the `def` keyword (see right).

```
# We first *define* our  
# function.  
def print_greeting(name):  
    print("Hello", name)  
  
# Later on in our code, we  
# *call* our function.  
print_greeting("Jack")  
print_greeting("Zainab")
```

# Some examples of useful functions

## From the math Library

- `math.floor()` # Round down
- `math.ceil()` # Round up
- `math.trunc()` # Truncation
- `math.factorial()` # Factorial
- `math.gcd()` # Greatest common divisor
- `math.pow()` # Power (exponentiation)
- `math.sin()` # Sine

## From shutil

- `shutil.copyfile()` # Copy files
- `shutil.move()` # Move files
- `shutil.rmtree()` # Delete files
- `shutil.make_archive()` # Zip files
- `shutil.unpack_archive()`  
# Unzip a file

# What are **arguments**?

- Arguments are values passed into a function when it is called.
- For example, the `input()` function takes 0 or 1 arguments which must be a string. We say it has **arity** 0 or 1.
- On the other hand, `print()` can take 0 or many arguments. It will take an unlimited amount of them! We say that it is **variadic**.
- How many arguments is `math.log()` being passed on the right? What value will be put in `x`?

```
# input() has arity 1 here.
num = input("Pick a number:")

# print() takes an unlimited
# number of arguments. In
# technical terms, it's
# *variadic*.
print(num, 1, 2, "Hello")

# How about math.log() here?
# What's its arity here?
x = math.log(2, 10)
```

# What are **return** values?

Functions may or may not generate a return value, depending on their purpose.

- Some functions, like `just_print()` on the right, do not return anything and do not take any arguments.
- Others, like `greet()` on the right take **parameters**, but don't return anything.
- Others still, like `triple()` on the right, take **parameters** and **return** something.

```
# No parameters, no return
# value.
def just_print():
    print("Message printed")

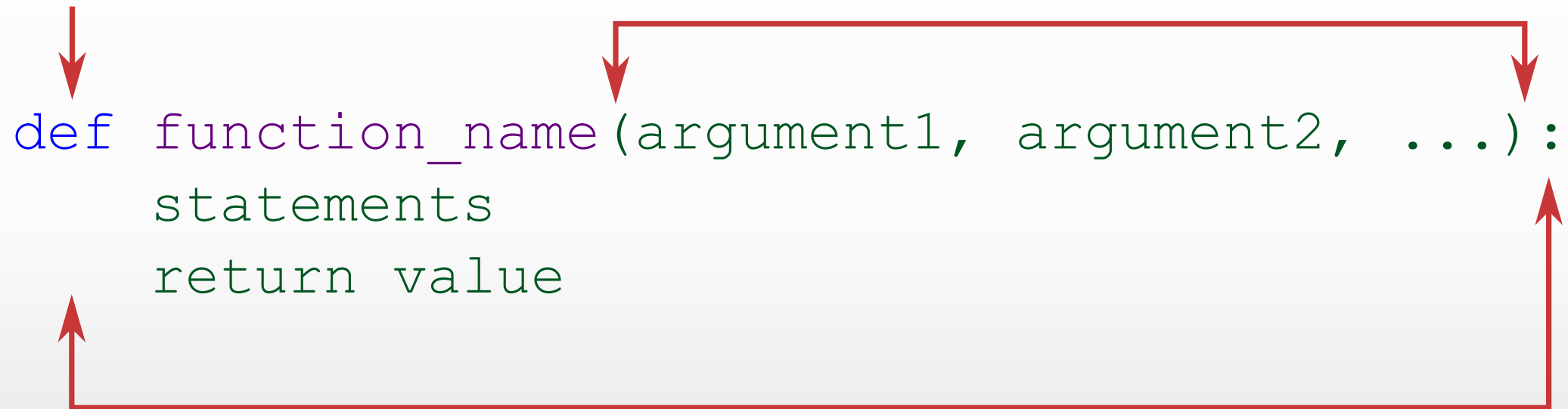
# Parameters, no return value.
def greet(name):
    print("Hello", name)

# Parameters and return value.
def triple(x):
    return x * 3
```

# The anatomy of a **function** definition

Keyword

Argument (parameter) list



```
def function_name(argument1, argument2, ...):  
    statements  
    return value
```

The diagram illustrates the components of a Python function definition. A red arrow points from the label 'Keyword' to the word 'def'. Another red arrow points from the label 'Argument (parameter) list' to the parentheses and contents '(argument1, argument2, ...)'. A third red arrow points from the label 'Where we have a colon, we need indentation!' to the colon at the end of the first line. A fourth red arrow points from the label 'Where we have a colon, we need indentation!' to the start of the indented lines 'statements' and 'return value'.

Where we have a colon, we need indentation!

# Invoking (calling) a function

- Once we've defined a **function**, we can invoke (or *call*) it. This means we transfer control to the function, which then executes its statements and returns its value (if any).
- We do this by simply stating the function name with brackets.

`function_name()`

- We might need to include arguments, or store the return value in a variable if we need it:

`my_var = function_name (arguments)`

```
# We first *define* our  
# function.
```

```
def print_greeting(name):  
    print("Hello", name)
```

```
# Later on in our code, we  
# *call* our function.
```

```
print_greeting("Jack")  
print_greeting("Zainab")
```

# function termination

- Functions can have more than one return statement, but can only ever return once.
- A function will terminate as soon as a return statement is encountered.
- Return values can have any type:
  - Lists
  - Numbers
  - Strings
  - Etc.

```
# The return here will  
# mean that the print  
# statement never runs!  
def useless_statement():  
    return 0  
    print("Never printed!")
```

# A **function** can take optional parameters

- When you define your own function in Python, you can make their parameters *optional*.
- This mean that if you don't pass an argument, it takes some value by default.
- The syntax for this is shown on the right, along with an example of some function calls.
- Notice that the parameter takes a default value of **"you"** if it isn't passed!

```
# Function with optional  
# parameter.  
def greet(name="you"):  
    print("Hello", name)  
  
# Prints "Hello James"  
greet("James")  
  
# Prints "Hello you"  
greet()
```

# A `function` can also be variadic

- We can define functions that take an unlimited number of arguments. We get these as a list in our function.
- The syntax for this is on the right.
- The `print()` function is an example of a function that is defined like this.
- I also encourage you to look at function definitions that look like this, though they are beyond the scope of this lecture:
  - `def myfunc(**kwargs)`
- Finally, function calls with *named arguments* that look like this:
  - `myFunc(z=3, x=5, y=6)`

```
# A variadic function.
def greet_many(*names):
    for name in names:
        print("Hello", name)

# Greet three people! Notice
# that we're passing 3
# arguments, which arrive in
# the function as a list.
greet_many("James", "Iris", "Ty")
```

# Passing by reference vs. Passing by value

- In Python, if you pass a list or object etc. to a function, and your function modifies it, the original thing will change! This is called *passing by reference*.
- There is an excellent explanation here involving coffee:  
<https://www.mathwarehouse.com/programming/passing-by-value-vs-by-reference-visual-explanation.php>



# Advanced lists

Lecture 3: Part 2

# Quick recap: Indexing lists

- We covered accessing list items (or, more technically, *members*) by index last session, but here's a quick refresher.
- List indices start at 0 and correspond to the position of each member of the list.
- Look at the code on the right, what does it print?
- Which statement here, if any, produces an error?

```
# Define a list.  
my_list = [7, 2, 8, 3, 6]  
  
print(my_list[1]) # This prints 2.  
  
print(my_list[0]) # This prints 7.  
  
# What does this print?  
print(my_list[4])  
  
# How about this?  
print(my_list[5])
```

# Reverse indices

- We can use reverse indices to go from the end of the list instead of the beginning.
- This is very useful for getting the last member of a list, for example.
- Remember, reverse indices start at **-1**, not **-0**, so to get the last element of a list called `my_list` we write `my_list[-1]`.
- Look at the calls on the right, what to they give us?

```
>>> my_list = [2, 7, 3, 9, 8]
```

```
>>> my_list[-1]  
8
```

```
>>> my_list[-4]  
7
```

```
>>> my_list[-6]  
# What will this give us?
```

# Deleting and inserting items

- We've looked at appending items to the end of lists, but what about inserting them into the middle?
- The `insert()` function comes to our rescue here (see right).
- Deleting items from lists by index uses a keyword in Python - `del`. For the syntax, see right.
- We can also use some math operators on lists! See right.

```
# Define a list.
myList = [7, 2, 8, 3, 6]

# Insert 100 at index 1: [7, 100, ...]
myList.insert(1, 100)

# Delete index 3: [7, 100, 2, 3, 6]
del myList[3]

# Add two lists together, then double
# the result up!
lx = [1, 2]
ly = [3, 4]
lz = (lx + ly) * 2

# lz == [1, 2, 3, 4, 1, 2, 3, 4]
```

# Slicing lists

- When we *slice* a list, we extract a portion of it.
- The general syntax for slicing a list looks like this:
  - `list[start:end:skip]`
- So to get elements 0, 1 and 2 from a list we do this:
  - `list[0:3]`
- Notice that start is *inclusive*, and end is *exclusive*. The element at the end index won't be included but the one at the first index will.
- We can use reverse indices with slicing!
- If we leave out indices, our slice goes to the start or end of the list by default.
- See the last line on the right for an example of skipping while slicing!

```
>>> myList = [1, 2, 3, 4, 5]

>>> myList[1:3]
[2, 3]

>>> myList[:4] # Default start index.
[1, 2, 3]

>>> myList[-3:-1] # Reverse indices.
[4, 5]

>>> myList[0:4:2]
[1, 3] # 0 to 4, jumping two each time!
```

# Useful list functions

```
>>> myList = [1, 2, 3] # Reversal!
>>> myList.reverse()
>>> myList
[3, 2, 1]

>>> myList = [45, 63, 11] # Sorting!
>>> myList.sort()
>>> myList
[11, 45, 63]

>>> myList = [1, 7, 2] # Find index!
>>> myList.index(7)
1
```

```
>>> myList = [1, 5, 6] # Sum!
>>> sum(myList)
12

>>> myList = [1, 5, 6] # Minimum!
>>> min(myList)
1

>>> myList = [1, 5, 6] # Maximum!
>>> max(myList)
6

>>> myList = [1, 6, 2, 7] # Length!
>>> len(myList)
4
```

# Any questions?

**Module Leader** Dr Alessandro Di Stefano

**Email** [A.DiStefano@tees.ac.uk](mailto:A.DiStefano@tees.ac.uk)

**Office** G0.44A (Greig building)

See Blackboard for online materials: <https://eat.tees.ac.uk>